

Spunti per introdurre alcuni concetti base dell'Intelligenza Artificiale in un liceo matematico

Stefano FINZI VITA (ex Matematica Sapienza)

Davide PASSARO (L. Russell, Roma)

Davide PALMIGIANI (ITIS e LSSA Vallauri, Velletri)

Motivazioni

- L'uso dell'IA si sta sempre più diffondendo nella società, creando un intenso dibattito a livello generale sulle sue potenzialità e sui suoi futuri ulteriori sviluppi, tra grandi aspettative e timore di catastrofiche conseguenze.
- Diventa perciò fondamentale fornire agli studenti delle conoscenze specifiche, permettendo loro di avere una chiave di lettura più profonda dell'IA. Anche per evitare di considerarla come un "oracolo" che fornisce risposte in modo "magico". Capirne le basi del funzionamento aiuta.
- I concetti base del funzionamento di una Rete Neurale (RN) e del Machine Learning (ML) possono essere spiegati sin dai primi anni del liceo.

Scaletta

- (S) **Introduzione generale:** Machine Learning (ML), Reti Neurali (RN), Large Language Models (LLM)
- (S) **Nozioni matematiche utilizzate** (alcuni richiami):
retropropagazione e discesa del gradiente
- (D1/D2) **Un percorso didattico per l'intero quinquennio**
(con varie declinazioni)
- (D1/D2) **Esempi di applicazione mediante coding in Python**
(Notebook Colab per il quinto anno)

L'idea del Machine Learning nasce da lontano

- **Alan Turing** (1947): "Vogliamo una macchina che possa imparare dall'esperienza, lasciandole la possibilità di modificare le proprie istruzioni per questo. Potremmo permetterle di dare anche risposte errate, ma in contatto con gli uomini potrà adattarsi ai loro standard."
- 1943: **McCulloch** e **Pitts** introducono l'idea del **neurone artificiale** sul modello di quello naturale
- 1957: **F. Rosenblatt** introduce il **perceptrone**: es. riconoscere nelle immagini cane/gatto
- 1969: **Minsky** e **Papert**: limiti del perceptrone e del singolo neurone artificiale, servono **reti neurali multi-strato**, ma non si sa come addestrarle: negli anni 70 frenata nella ricerca
- 1986: **Rumelhart**, **Hinton** e **Williams** propongono un algoritmo di **retropropagazione** degli errori per addestrare le reti
- 2017 **A. Vaswami et al.**: introducono l'architettura **Transformer**, alla base dei modelli di **Large Language Models**

(Treccani) **Machine Learning**: branca dell'IA che si occupa dello sviluppo di algoritmi e tecniche finalizzate all'apprendimento automatico mediante statistica computazionale e ottimizzazione matematica.

Applicazioni (alcuni esempi):

- riconoscimento di messaggi email spam
- riconoscimento ottico di caratteri (lettere/numeri)
- ricerca per immagini
- riconoscimento facciale
- riconoscimento del parlato (assistenti virtuali)
- suggerimento di prodotti / estrazione di opinioni
- veicoli a guida autonoma
- sintesi di immagini, suoni o altre forme di dati
- diagnostica medica assistita

LLM: una nuova risorsa a disposizione

L'idea di fondo dell'articolo di Vaswami del 2017 ([Attention is all you need](https://arxiv.org/abs/1706.03762), <https://arxiv.org/abs/1706.03762>) è stata l'introduzione di un'architettura di reti neurali nominata **Transformer**, basata su di una struttura in grado di elaborare e comprendere i dati in modo più efficiente

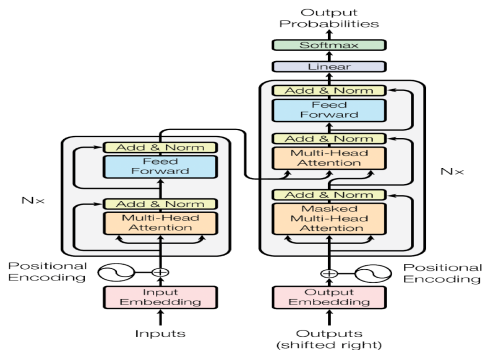


Figure 1: The Transformer - model architecture.

Cos'è un Large Language Model? (ChatGPT)

*Un **LLM** (Large Language Model) è un modello di intelligenza artificiale addestrato su enormi quantità di testo per comprendere e generare linguaggio naturale. Utilizza reti neurali profonde per prevedere parole, rispondere a domande e svolgere attività linguistiche complesse, come la scrittura e la traduzione.*

Tu sei quindi un LLM?

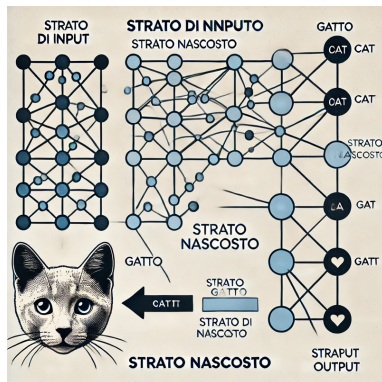
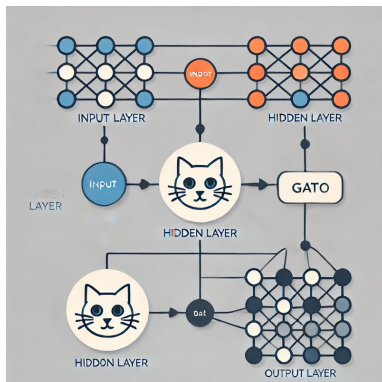
Sì, esatto! Sono un LLM sviluppato da OpenAI. Uso reti neurali avanzate per comprendere il linguaggio naturale, rispondere alle domande, generare testi e aiutarti con una vasta gamma di argomenti. In pratica, sono come un assistente virtuale super addestrato sul linguaggio!

Che cos'è una Rete Neurale?

Definizione (ChatGPT):

*Una **Rete Neurale** è un tipo di sistema informatico che cerca di imitare il modo in cui funziona il cervello umano per risolvere problemi. È composta da tanti "nodi" (chiamati anche neuroni artificiali) collegati tra loro. Questi nodi ricevono informazioni, le elaborano e poi inviano i risultati agli altri nodi, proprio come i neuroni nel cervello trasmettono segnali. Questi sistemi vengono usati per compiti come il riconoscimento delle immagini, la traduzione delle lingue o anche la previsione del tempo. La rete neurale "impara" dai dati che gli vengono dati, proprio come una persona impara con l'esperienza. Più dati riceve, più diventa brava nel compito che le è stato assegnato.*

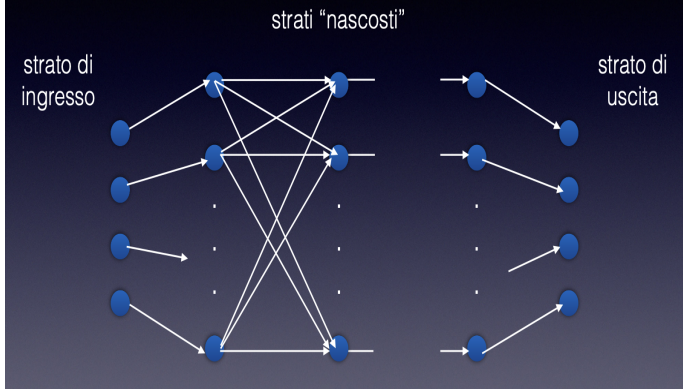
Ho chiesto a ChatGPT di fornirmi un'immagine in grado di descrivere con semplicità il funzionamento di una RN. Poi ho chiesto di tradurre i titoli in italiano. Ecco i risultati:



'Non sei un buon traduttore!'

'A volte l'IA non è perfetta con le scritte nelle immagini!'

Reti neurali multi-strato



(da [V. Bonifaci](#) (RM 3): corso di Machine Learning)

Come funziona una RN? (ChatGPT)

1. **Input:** La rete neurale riceve delle informazioni iniziali (per es. numeri, immagini o un testo).
2. **Elaborazione:** Ogni "neurone" della rete elabora i dati ricevuti, combinando le informazioni in base a delle regole. Questo avviene in vari "strati" (input, nascosti e output).
3. **Pesi e Bias:** Ogni connessione tra i neuroni ha un "peso" che determina quanto essa è importante. La rete cerca di regolare questi pesi per migliorare i suoi risultati. In più, ci sono i "bias", che aiutano a regolare l'output del neurone.
4. **Propagazione in avanti:** L'informazione attraversa i vari strati nascosti, subendo trasformazioni per affinare il risultato.
5. **Output e apprendimento:** Alla fine, la rete produce una risposta. Se è sbagliata, si corregge modificando i pesi e i bias grazie a un processo chiamato **backpropagation**, basato su errori che confrontano il risultato ottenuto con quello desiderato.

Così, con il tempo e tanti dati, la rete diventa sempre più precisa.

E la matematica in tutto questo dove sta? (ChatGPT)

La matematica è **il cuore** di tutto! Sia le RN che i LLM si basano su concetti matematici fondamentali. Vediamo dove entra in gioco:

- **Algebra Lineare**: i dati (testo, immagini, numeri) vengono trasformati in **vettori**; le operazioni tra neuroni sono moltiplicazioni tra **matrici e vettori**.
- **Probabilità e Statistica**: le reti neurali si allenano ottimizzando distribuzioni di probabilità per prendere decisioni migliori.
- **Calcolo Differenziale**: durante l'apprendimento, usiamo la **discesa del gradiente** per aggiornare i pesi e migliorare il modello; la **backpropagation** calcola la derivata dell'errore rispetto ai pesi per capire come modificarli.
- **Ottimizzazione**: i modelli vengono addestrati minimizzando una **funzione di perdita**, come l'errore tra output e risultato atteso.

In sintesi, la matematica è **la colonna portante** che permette a questi modelli di imparare e migliorare. Senza algebra, probabilità e calcolo, non esisterebbero né reti neurali né LLM!

Dall'Introduzione del libro di [A. Ananthaswami](#), *Perché le macchine imparano. L'eleganza della matematica dietro all'AI*, Apogeo 2024:

- Le macchine possono imparare grazie alla straordinaria sinergia fra matematica e informatica, con l'aggiunta di un pizzico di fisica e neuroscienza.
- Quella su cui si basa il ML è una matematica relativamente semplice ed elegante, che si impara alle superiori o all'inizio dell'università. Ma tutto questo senza il progresso dell'informatica dell'ultimo decennio sarebbe stato inimmaginabile.
- Comprendere il funzionamento dell'IA è fondamentale per capire non solo la potenza della tecnologia, ma anche i suoi limiti.
- Non possiamo lasciare solo ai suoi ideatori le decisioni su come l'IA sarà costruita e impiegata. Per regolamentare efficacemente questa tecnologia utile ma potenzialmente minacciosa, altri strati della società (educatori, legislatori, comunicatori scientifici ma anche semplici utenti) devono affrontare la matematica che ci sta sotto.

Un esempio elementare di modello da 'addestrare'

- Considerate la seguente tabella:

x_1	x_2	$\rightarrow y$
4	2	8
1	2	5
0	5	10
2	1	4

e poi completate correttamente la seguente sequenza:

3 2 ??

Un esempio elementare di modello da 'addestrare'

- Considerate la seguente tabella:

x_1	x_2	$\rightarrow y$
4	2	8
1	2	5
0	5	10
2	1	4

e poi completate correttamente la seguente sequenza:

3 2 ??

- Il modello è: $y = x_1 + 2x_2$, quindi la risposta è: 7
Noi ci siamo arrivati con la logica, le macchine seguono un'altra strada

Da uno strato all'altro: l'attivazione

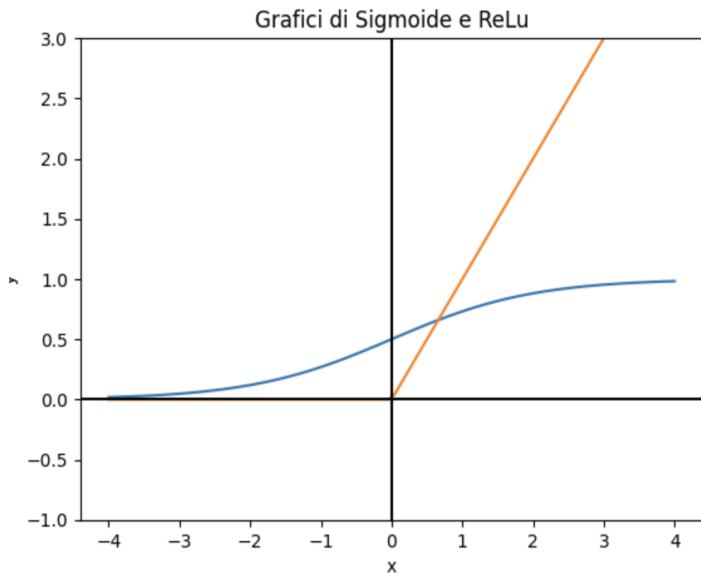
- La rete ha più strati, quello di ingresso (**input**), quello d'uscita (**output**) e altri strati intermedi tra loro (in genere nascosti).
- Ogni strato k ha un certo numero di neuroni: $x_1^k, \dots, x_n^k$
- Il passaggio da uno strato al successivo avviene attraverso una funzione di attivazione che utilizza una combinazione dei neuroni mediante pesi e bias; ad es. il primo neurone dello strato $k + 1$ sarà

$$x_1^{k+1} = \sigma(w_1^k x_1^k + \dots + w_n^k x_n^k + b_1^k)$$

- Tipiche funzioni di attivazione (non lineari):

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (\textit{sigmoide})$$

$$\textit{ReLU}(x) = \max(x, 0) \quad (\textit{RectifiedLinearUnit})$$



L'apprendimento della rete neurale

- Si parte da un esempio test (X, T) , dove $X = \{x_i\}$ è il vettore degli input e $T = \{t_i\}$ quello dei valori attesi; con pesi iniziali random, si ottiene un output $Y = \{y_i\}$, si calcola il costo, cioè l'errore rispetto al valore atteso, per esempio attraverso lo **scarto quadratico medio**

$$C(Y) = \frac{1}{2} \sum_i (y_i - t_i)^2$$

Ovviamente Y (e quindi C) dipendono dai neuroni dei diversi strati e dai pesi e bias utilizzati per ogni passaggio da uno strato all'altro.

- Lo scopo della **Backpropagation** è quello di ridurre il costo modificando i parametri utilizzati: in pratica partendo dall'errore finale si risale all'indietro mediante la strategia di **discesa del gradiente**.
- La procedura si ripete sull'intero set dei test di apprendimento, fino a che l'errore finale risulta accettabile. Più test si provano, più la rete impara, più sarà pronta alla fine per valutare delle situazioni nuove.

La discesa del gradiente: il caso 1D

- Se vogliamo ridurre il valore di una funzione data $f(x)$ partendo da una certa posizione $(x_0, f(x_0))$ dove ci troviamo, ci serve valutarne la pendenza, cioè in quale direzione spostarsi per scendere. Queste informazioni ce le dà la derivata di f .
- Ad esempio se f è il grafico di una retta ($f(x) = mx + q$) sappiamo che m indica il coefficiente angolare, e quindi la pendenza della retta. Quindi per scendere ci sposteremo a sinistra di x_0 se $m > 0$, a destra se $m < 0$. In pratica nella direzione opposta a quella di crescita.
- In generale, se f è una funzione derivabile, ci sposteremo nella direzione opposta a quella individuata dal segno della derivata $f'(x_0)$:

$$x_1 = x_0 - \alpha f'(x_0)$$

con α una costante per regolare la lunghezza del passo.

L'algoritmo di Discesa del Gradiente nel caso convesso

Sia $f : D \rightarrow \mathbb{R}$ una funzione convessa e derivabile. Allora f ammette un minimo globale x^* ($f(x^*) \leq f(x)$ per ogni $x \in D$) (con $f'(x^*) = 0$), che si può approssimare mediante il seguente algoritmo:

- ① Per $k = 1, \dots, N$: $x_{k+1} = x_k - \alpha f'(x_k)$
- ② Restituisci x_k col minimo valore di $f(x_k)$:

$$x_G = \operatorname{argmin}_{k=1, \dots, N} f(x_k)$$

Vale il seguente risultato di convergenza:

Teorema. Se f è convessa e derivabile, ed esistono due costanti $D, G > 0$ tali che:

$$\|x_1 - x^*\| \leq D$$

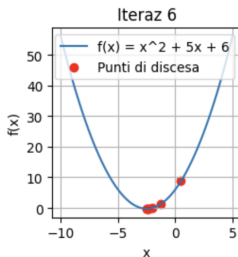
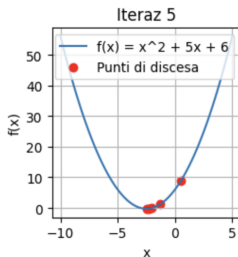
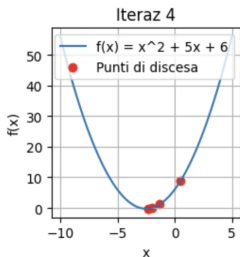
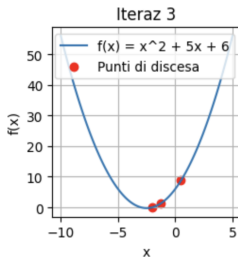
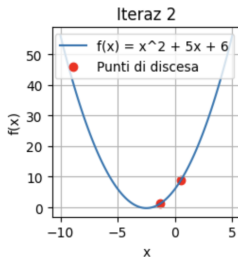
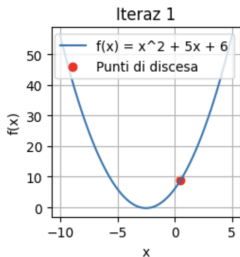
$$\|f'(x_k)\| \leq G \text{ per } k = 1, \dots, N$$

allora per $\alpha = D/(G\sqrt{N})$ il metodo soddisfa

$$f(x_G) - f(x^*) \leq \frac{DG}{\sqrt{N}}$$

Esempio: una parabola ($x_0 = 5$, $\alpha = 0.3$, $x^* = -2.5$)

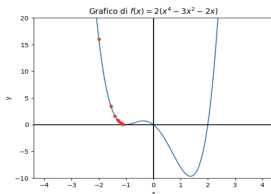
Discesa del gradiente



-2.46928

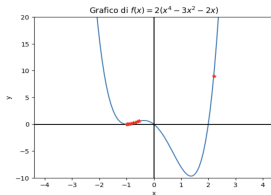
Nell'esempio precedente le cose vanno bene perché la funzione è regolare, convessa, con un unico punto di minimo. Anche variando α la sequenza convergerà sempre al minimo, magari non più in modo monotono. Nei casi in cui la funzione ha più valori di minimo locale, la scelta del parametro α diventa invece cruciale.

gradiente(-2,0,01,0)



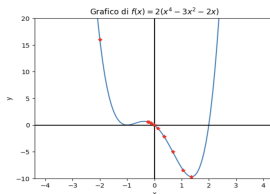
-1.1007528189951934

gradiente(2,2,0,01,0)



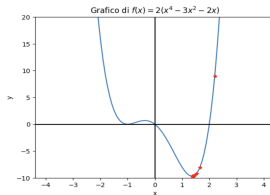
-0.990422676636388

gradiente(-2,0,04,0)



1.3401300923250037

gradiente(2,2,0,01,0)



1.3741622295655105

La discesa del gradiente: il caso 2D

Sia ora $f(x, y)$ la funzione da minimizzare. Ora non c'è una derivata sola, ma un vettore delle derivate parziali, cioè il gradiente di f , che indica la direzione di massima ascesa in ogni punto della superficie:

$$\nabla f(x, y) = \left[\frac{\partial f}{\partial x}(x, y), \frac{\partial f}{\partial y}(x, y) \right]$$

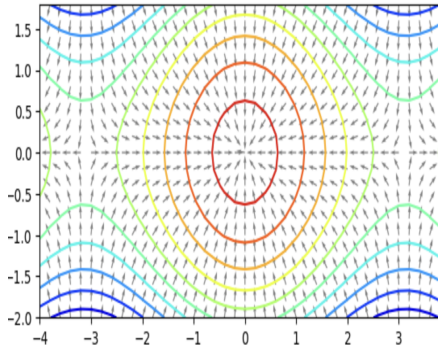
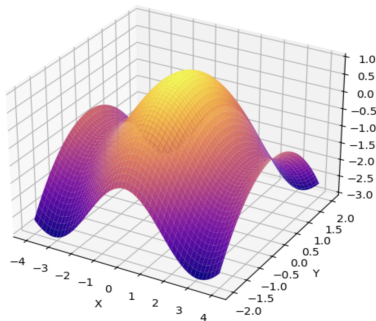
Quindi per scendere occorre andare nella direzione opposta. Se (x_0, y_0) è la posizione iniziale, allora

$$(x_1, y_1) = (x_0, y_0) - \alpha \nabla f(x_0, y_0) = (x_0 - \alpha \frac{\partial f}{\partial x}(x_0, y_0), y_0 - \alpha \frac{\partial f}{\partial y}(x_0, y_0))$$

In altre parole la direzione del gradiente è sempre perpendicolare alle curve di livello della funzione.

Esempio di curve di livello: $f(x, y) = \cos(x) - \frac{1}{2}y^2$

Grafico di $z = \cos(x) - 0.5y^2$



La discesa del gradiente: il caso delle reti

- In una rete neurale la funzione da minimizzare ha tantissime variabili, ma il procedimento è lo stesso. Il costo dipende dai neuroni dell'output, che dipendono a loro volta da quelli degli strati precedenti e da tutti i pesi e bias utilizzati per l'avanzamento. Quindi parliamo di funzioni composte che andranno derivate con la **chain rule**.
- Se per es. $y = f(x)$, $x = g(z)$, allora $y = F(z) = f(g(z))$ e quindi, se entrambe le funzioni sono derivabili:

$$\begin{aligned}
 F'(z) &= \lim_{t \rightarrow z} \frac{F(t) - F(z)}{t - z} = \lim_{t \rightarrow z} \frac{f(g(t)) - f(g(z))}{t - z} \\
 &= \lim_{t \rightarrow z} \frac{f(g(t)) - f(g(z))}{g(t) - g(z)} \frac{g(t) - g(z)}{t - z} \\
 &= f'(g(z)) \cdot g'(z)
 \end{aligned}$$

La discesa del gradiente: il caso delle reti

- Immaginiamo una rete neurale molto semplice con input, uno strato nascosto e output, tutti di un solo neurone: (x, a, y) ; allora

$$a = \sigma(z_1) = \sigma(w_1x + b_1), \quad y = \sigma(z_2) = \sigma(w_2a + b_2), \quad C = (y - t)^2$$

$$\frac{\partial C}{\partial w_2} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial z_2} \frac{\partial z_2}{\partial w_2} = 2(y - t)\sigma'(z_2)a$$

$$\frac{\partial C}{\partial w_1} = \frac{\partial C}{\partial y} \frac{\partial y}{\partial z_2} \frac{\partial z_2}{\partial a} \frac{\partial a}{\partial z_1} \frac{\partial z_1}{\partial w_1} = 2(y - t)\sigma'(z_2)w_2\sigma'(z_1)x$$

- Lo stesso vale per le derivate del costo rispetto ai bias, tenendo conto che

$$\frac{\partial z_2}{\partial b_2} = \frac{\partial z_1}{\partial b_1} = 1$$

La discesa del gradiente: il caso generale

- Generalizzare quanto appena visto non è impossibile, ma richiede un grosso appesantimento delle formule: per tener conto di tutti i parametri dovremmo aggiungere molti indici. Per fortuna ci aiuta l'algebra lineare.
- Per esempio il passaggio da uno strato all'altro lo potremmo scrivere in modo compatto come $X^{k+1} = \sigma(W^k X^k + B^k)$, dove

$$W^k = \begin{bmatrix} w_{11}^k & w_{12}^k & \cdots & w_{1n}^k \\ w_{21}^k & w_{22}^k & \cdots & w_{2n}^k \\ \cdots & \cdots & \cdots & \cdots \\ w_{n1}^k & w_{n2}^k & \cdots & w_{nn}^k \end{bmatrix} \quad X^k = \begin{bmatrix} x_1^k \\ x_2^k \\ \cdots \\ x_n^k \end{bmatrix} \quad B^k = \begin{bmatrix} b_1^k \\ b_2^k \\ \cdots \\ b_n^k \end{bmatrix}$$

- Analogamente per il calcolo del gradiente della funzione costo le catene delle derivate si allungherebbero di molto. Non entriamo nei dettagli, ma è facile immaginare come anche questo si possa formulare attraverso prodotti di matrici per vettori.

Bibliografia e sitografia (il mio 'training')

- A. Ananthaswamy, [Perché la macchine imparano. L'eleganza della matematica dietro all'AI](#), Apogeo, 2024
- D. Palmigiani e D. Passaro, [ChatGPT e insegnamento della matematica](#), Archimede: 2, 2023
- D. Palmigiani e D. Passaro, [Alcuni concetti chiave dell'Intelligenza Artificiale per favorirne un uso consapevole](#), Archimede: 3, 2024
- D. Palmigiani e D. Passaro, [Concetti di intelligenza artificiale a scuola: introduzione al deep learning](#), Archimede: 2025, in via di pubblicazione

- A. Quarteroni, [Scientific ML. Quando l'IA incontra le Scienze Fondamentali](#), Accademia dei Lincei, 13.12.2024,
https://www.youtube.com/watch?v=zTrGEa02_3o
- Grant Sanderson, [Neural Networks. The basics of neural networks, and the math behind how they learn](#), disponibile al link:
<https://www.3blue1brown.com/topics/neural-networks>
- V. Bonifaci (Univ. Roma Tre), [Discesa del gradiente per l'apprendimento supervisionato](#), slides del corso di Machine Learning,
<https://ricerca.mat.uniroma3.it/users/vbonifaci/in550.html>

Ma ora la parola agli esperti...